# What Are Convolutional Neural Networks? A Complete CNN Guide

*Condensed from the DataCamp tutorial by Zoumana Keita (Mar 26, 2026), with a hands-on coding section added.*

Convolutional neural networks power some of today's most impressive AI capabilities, from facial recognition on smartphones to tumor detection in medical imaging. This guide covers what CNNs are, how they work, their key components, strategies to combat overfitting, practical applications, popular architectures, frameworks, and finally **working code** to build and train one.

## TL;DR

- A CNN is a deep learning architecture designed for image classification, object detection, and segmentation.
- CNNs have four key components: convolutional layers (feature extraction), activation functions like ReLU (non-linearity), pooling layers (dimensionality reduction), and fully connected layers (classification).
- Their design is inspired by the hierarchical structure of the human visual cortex.
- Overfitting is a major challenge; dropout, batch normalization, data augmentation, and early stopping help mitigate it.
- TensorFlow, PyTorch, and Keras are the most popular frameworks for building CNNs.

## 1. What is a Convolutional Neural Network?

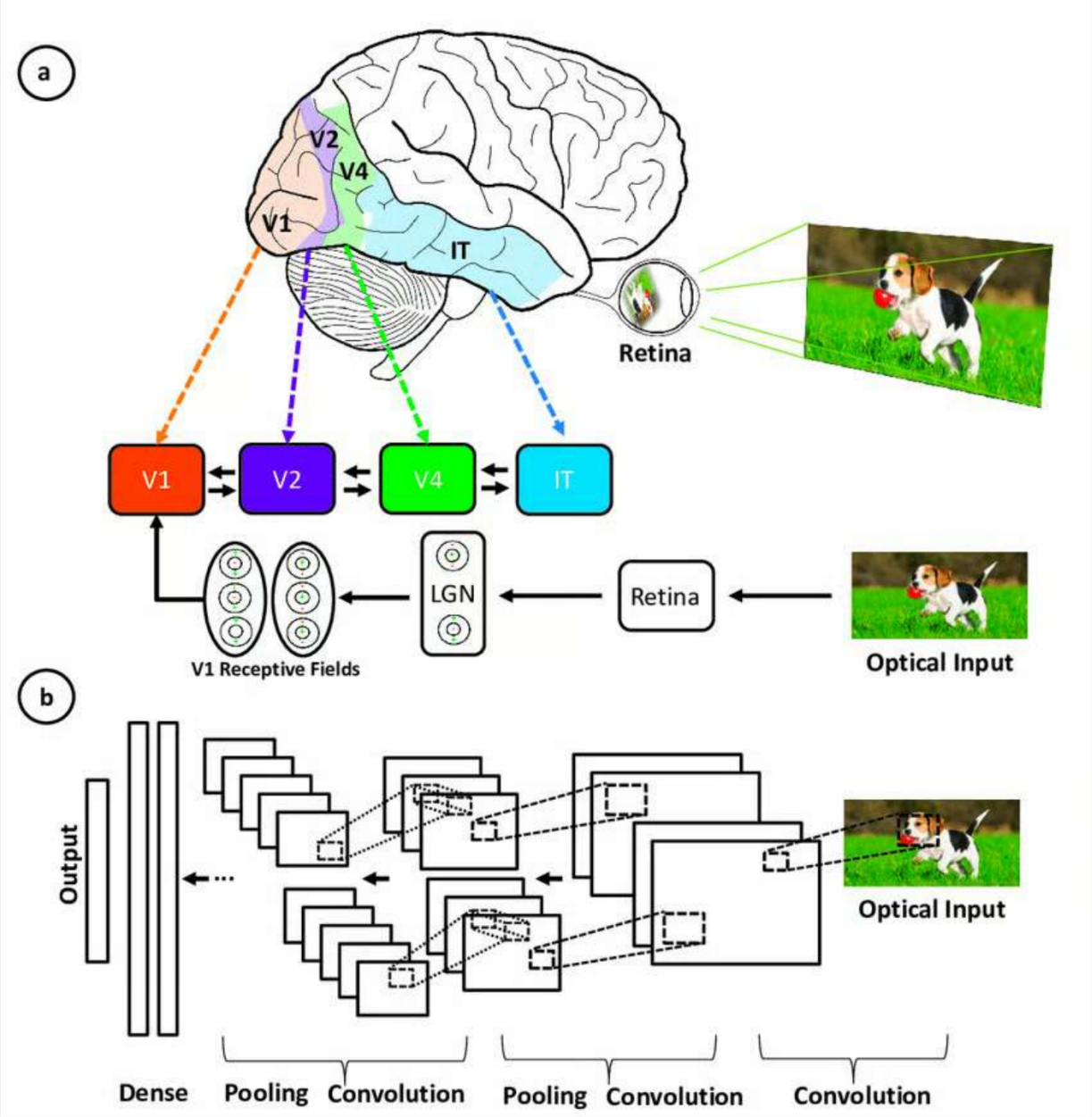
A Convolutional Neural Network (CNN), also known as ConvNet, is a specialized type of deep learning algorithm mainly designed for tasks that need object recognition, including image classification, detection, and segmentation. CNNs are used in autonomous vehicles, security camera systems, and more.

### Why CNNs matter

- Unlike classic algorithms such as SVMs and decision trees, CNNs extract features automatically at scale, bypassing manual feature engineering.
- Convolutional layers give CNNs translation-invariant characteristics: they identify patterns regardless of position, orientation, scale, or translation.
- Pre-trained architectures (VGG-16, ResNet50, Inceptionv3, EfficientNet) perform at top level and can be adapted to new tasks with little data through fine-tuning.
- Beyond images, CNNs apply to natural language processing, time series analysis, and speech recognition.

## 2. Inspiration From the Human Visual System

CNNs were inspired by the layered architecture of the human visual cortex. Key similarities:




*Correspondence between areas of the primary visual cortex and the layers of a CNN.*

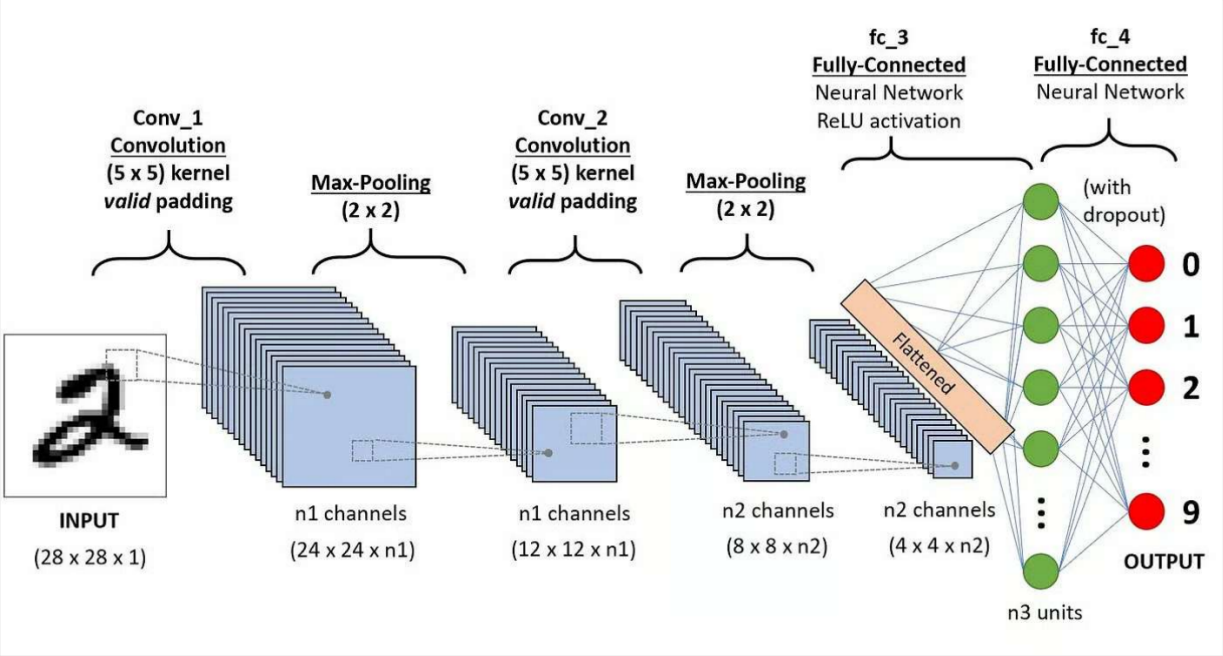
- **Hierarchical architecture:** simple features are extracted in early layers and more complex ones in deeper layers.
- **Local connectivity:** neurons connect only to a local region of the input, which makes the network efficient.

- **Translation invariance:** visual neurons detect features regardless of location; pooling layers give CNNs a degree of this.
- **Multiple feature maps:** many feature maps per stage, mimicked by multiple filters in each convolution layer.
- **Non-linearity:** achieved with activation functions such as ReLU after each convolution.

CNNs are simpler than the human visual system: they lack its complex feedback mechanisms and rely on supervised rather than unsupervised learning.

# 3. Key Components of a CNN

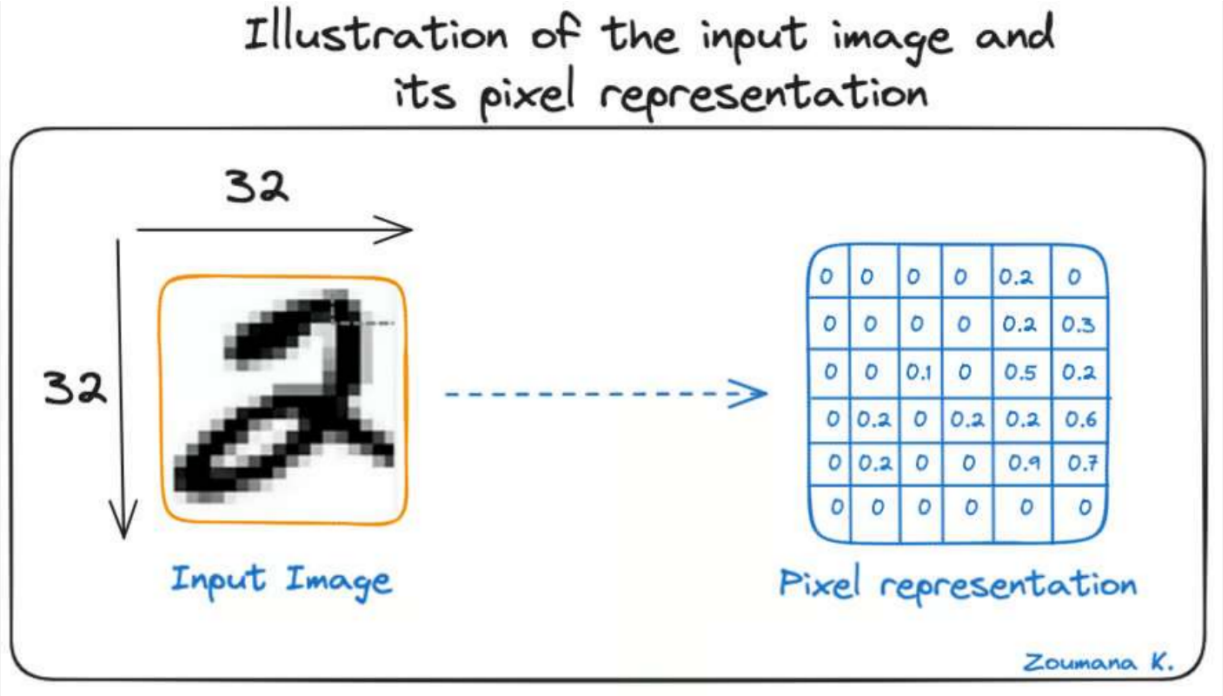
A CNN is made of four main parts: convolutional layers, ReLU, pooling layers, and fully connected layers. The example below classifies a handwritten digit.




*Architecture of a CNN applied to digit recognition.*

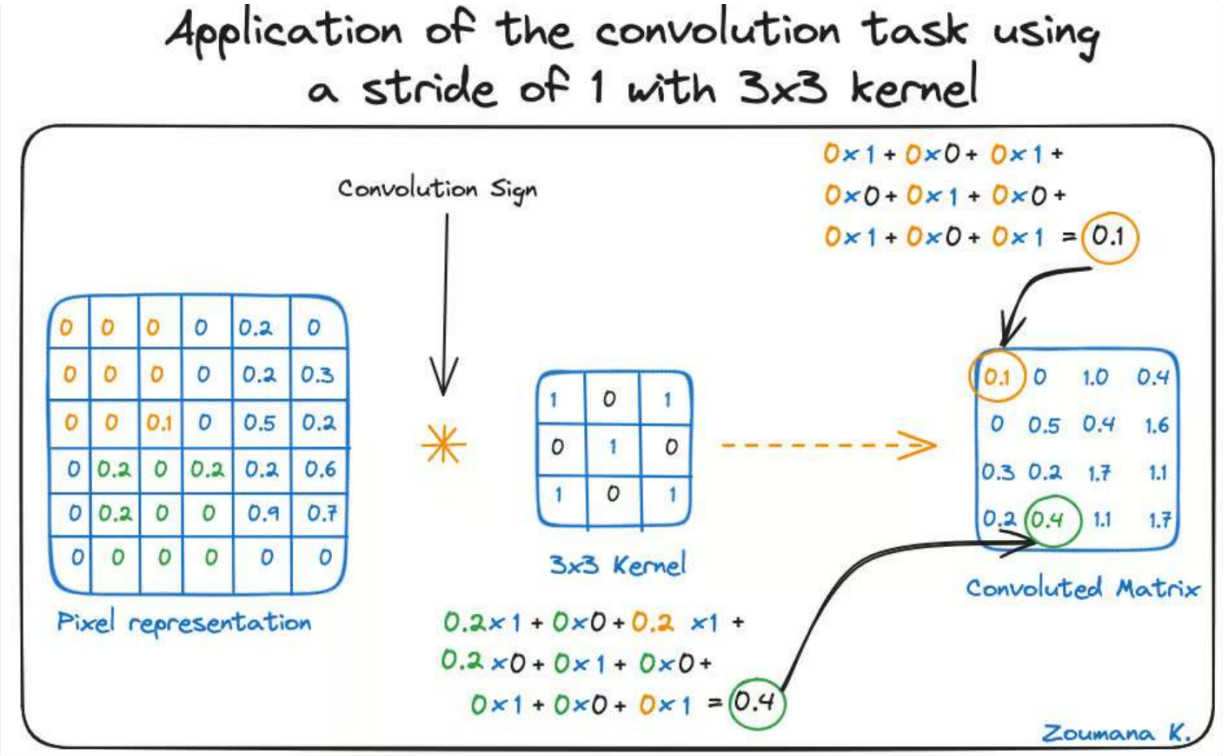
## Convolution layers

The main mathematical task is convolution: sliding a small matrix (the *kernel* or *filter*) over the pixel matrix of an image. Each filter detects a specific pattern, such as lines, curves, or edges. Several filters together give the network a good picture of all the patterns that make up the image.




*An input image and its pixel representation.*

The convolution works as follows: place the kernel at the top-left corner, multiply element-wise, sum the products, and store the result as the first value of the output matrix. Then slide the kernel (by the stride) and repeat until the image is covered. A larger stride gives a smaller output. In practice, kernel weights are learned during training, not set by hand.




*Convolution with a stride of 1 and a 3x3 kernel.*

The kernel is also called a *feature detector*. For instance, an averaging kernel blurs an image, while a subtracting kernel detects edges. The more convolution layers a network has, the more abstract the features it can detect.

## Activation function

A ReLU activation is applied after each convolution. It helps the network learn non-linear relationships between features, makes it more robust at identifying different patterns, and helps mitigate vanishing gradients.

## Activation Function in Deep Learning

**Activation Function** হলো Neural Network-এর এমন একটি function, যা কোনো neuron-এর weighted sum ও bias থেকে output নির্ধারণ করে। এটি Neural Network-এ **non-linearity** যোগ করে, ফলে মডেল জটিল pattern ও relationship শিখতে পারে।

### 1. Activation Function কীভাবে কাজ করে?

একটি neuron-এর প্রথম ধাপ হলো input-এর weighted sum হিসাব করা:

$$z = w_1x_1 + w_2x_2 + \cdots + w_nx_n + b$$

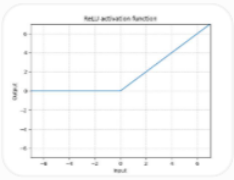
এরপর activation function প্রয়োগ করা হয়:

$$\boxed{a = f(z)}$$

যেখানে:

- $x$ = Input features
- $w$ = Weights
- $b$ = Bias
- $z$ = Weighted sum
- $f(z)$ = Activation function
- $a$ = Neuron-এর output

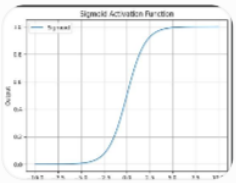
## 2. Types of Activation Functions



### 1. ReLU (Rectified Linear Unit)

$$f(x) = \max(0, x)$$

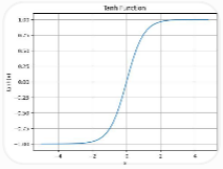
- $x < 0$ হলে output = 0
- $x > 0$ হলে output = $x$
- Hidden layers-এ বহুল ব্যবহৃত।




### 2. Sigmoid

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

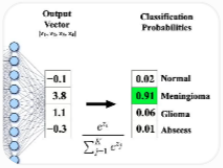
- Output: $0$ থেকে $1$
- Binary classification-এর output layer-এ ব্যবহৃত হয়।
- বড় positive বা negative input-এ gradient খুব ছোট হয়ে যেতে পারে।



### 3. Tanh

$$f(x) = \tanh(x)$$

- Output: $-1$ থেকে $1$
- Output zero-centered।
- বড় input-এর ক্ষেত্রে gradient ছোট হওয়ার সমস্যা হতে পারে।




### 4. Softmax

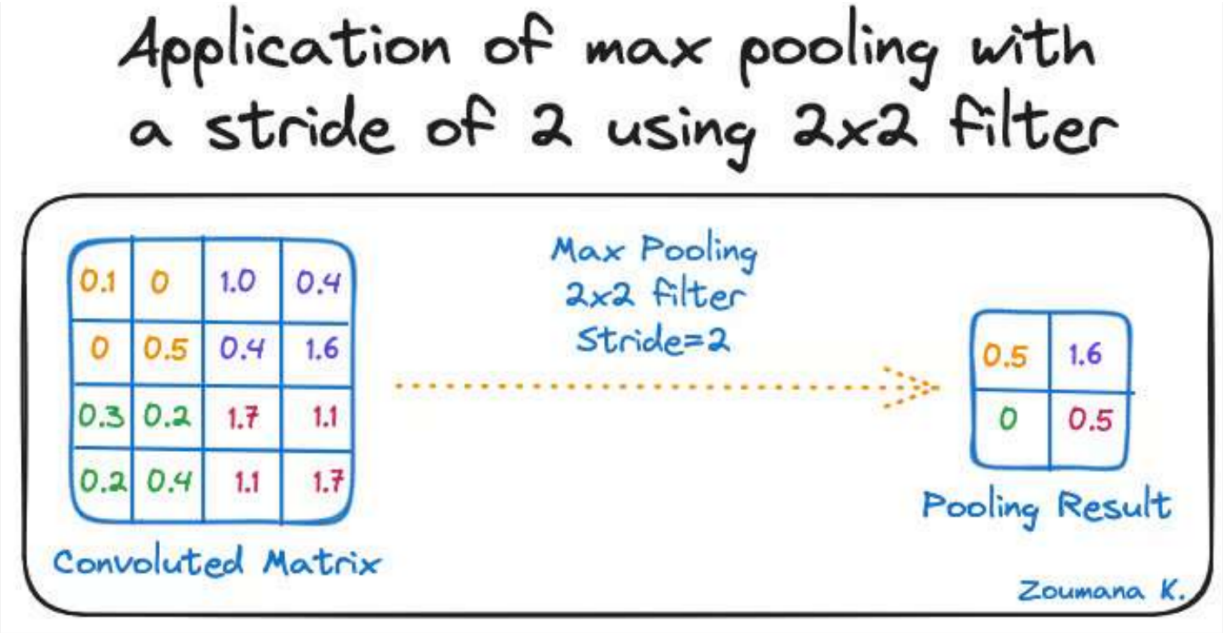
$$f(z_i) = \frac{e^{z_i}}{\sum_j e^{z_j}}$$

- একাধিক class-এর score-কে probability distribution-এ রূপান্তর করে।
- Multiclass classification-এর output layer-এ ব্যবহৃত হয়।
- সব class-এর probability-এর যোগফল 1।

*Common activation functions: ReLU, Sigmoid, Tanh and Softmax.*

## Pooling layer

Pooling pulls the most significant features from the convolved matrix, reducing the feature map size (and memory use) and helping reduce overfitting. Common choices: **max pooling** (maximum value), **sum pooling**, and **average pooling**.




*Max pooling with a stride of 2 and a 2x2 filter.*

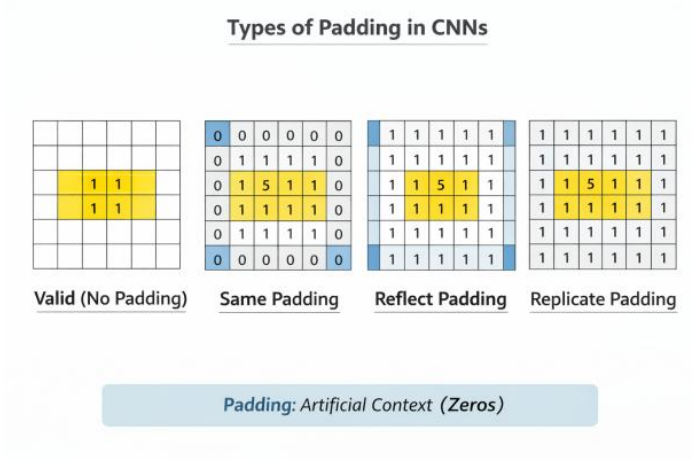
## Padding and strides

**Padding** and **stride** are two important convolution parameters. They control the size of the output feature map, how much spatial information is kept, and how much computation is needed.

- **Padding:** adding extra pixels (usually zeros) around the border of the input image.
- **Stride:** how many pixels the filter moves at each step.

**Why padding?**

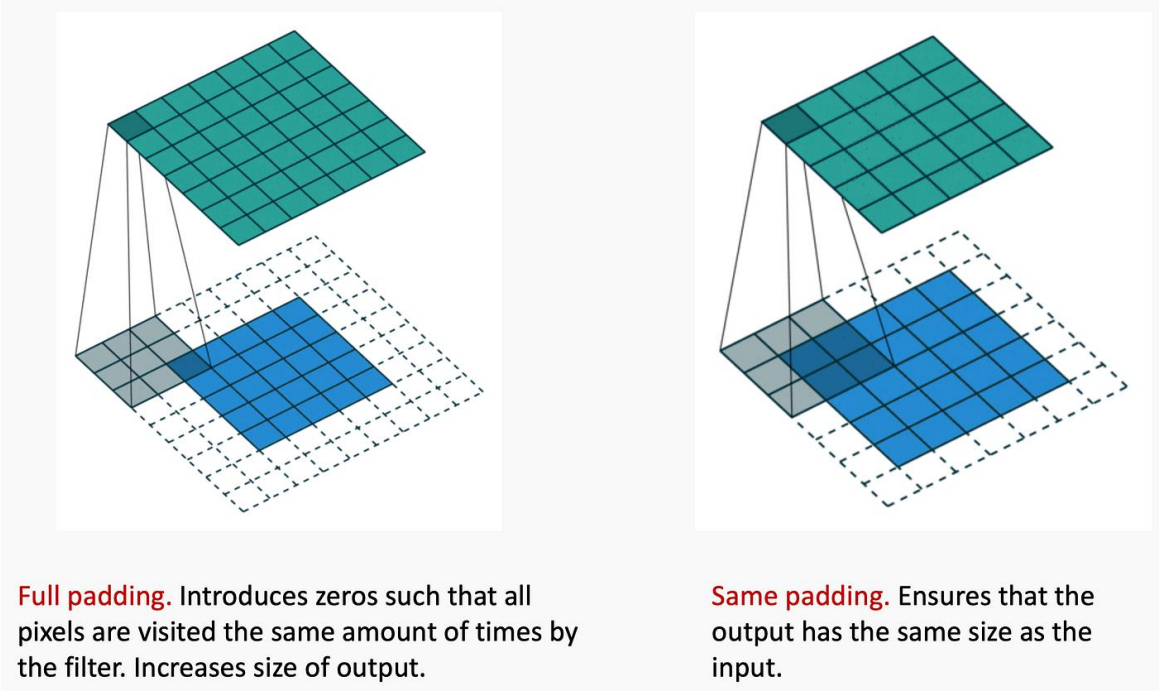
Without padding, edge pixels are covered by fewer filter positions than centre pixels, so border information is only partially processed, and the output is smaller than the input. Every convolution layer would shrink the feature map. Padding gives the border pixels more coverage and lets you control the output size.



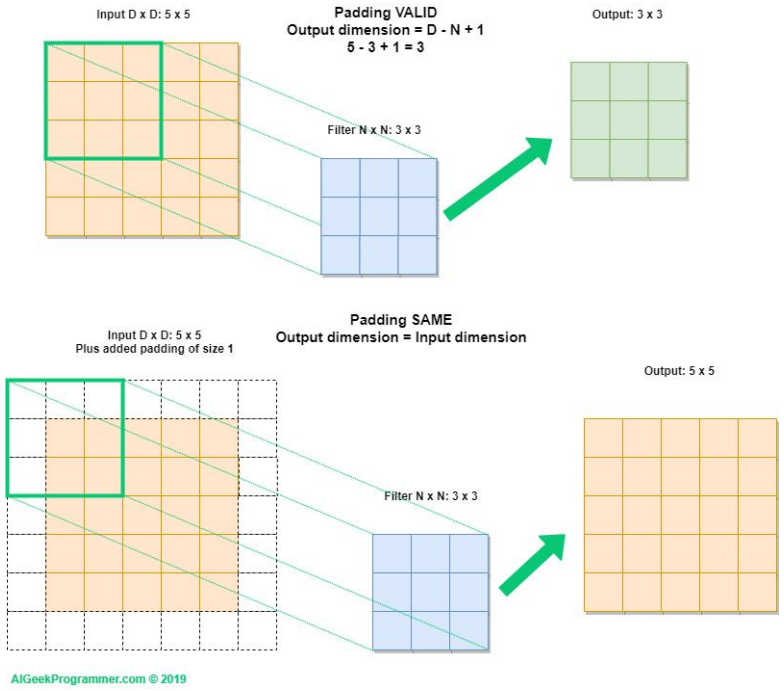

*Types of padding: valid (none), same (zeros), reflect, and replicate.*

**Common padding types:**

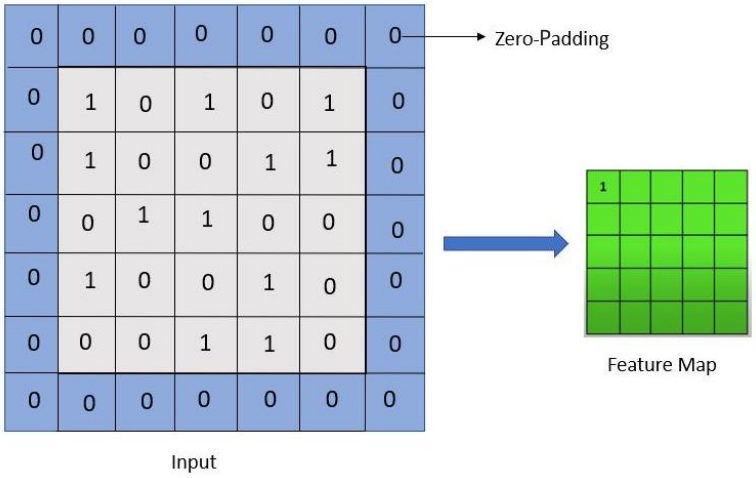
- **Valid:** no padding. The output shrinks to (W - F + 1).
- **Same:** zero padding chosen so the output has the same size as the input (for stride 1).
- **Full:** pads so that every input pixel is visited the same number of times by the filter; the output is larger than the input.
- **Reflect / replicate:** instead of zeros, the border is filled by mirroring or repeating edge pixels (available in PyTorch via `padding_mode`).




*Full padding (left) and same padding (right).*




*Valid vs. same padding on a 5x5 input with a 3x3 filter: 3x3 output vs. 5x5 output.*



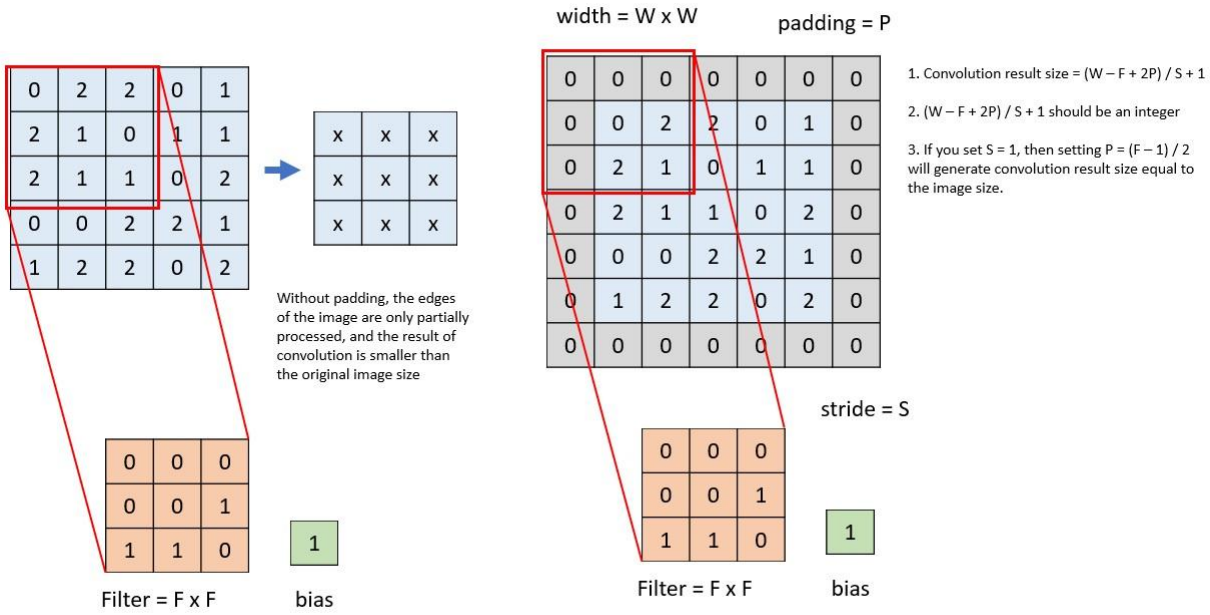

*A 5x5 input with a one-pixel border of zero padding.*

### Output size formula

For an input of width W, filter size F, padding P and stride S:

```
output size = floor( (W - F + 2P) / S ) + 1

Stride 1 and "same" output:  P = (F - 1) / 2        (F = 3 -> P = 1, F = 5 -> P = 2)
```




*Without padding the edges are only partly processed (left); with padding P the output keeps the input size (right).*

Worked examples for a 5x5 input and a 3x3 filter (all verified by running the code below):

| Padding | Stride | Calculation | Output |
|---|---|---|---|
| 0 (valid) | 1 | (5 - 3 + 0)/1 + 1 | 3x3 |
| 1 (same) | 1 | (5 - 3 + 2)/1 + 1 | 5x5 |
| 0 | 2 | floor((5 - 3 + 0)/2) + 1 | 2x2 |
| 1 | 2 | floor((5 - 3 + 2)/2) + 1 | 3x3 |

**Stride (S)** is the number of pixels the filter moves after each operation. It controls how far the filter jumps, and so how big the output is and how much computation is needed.

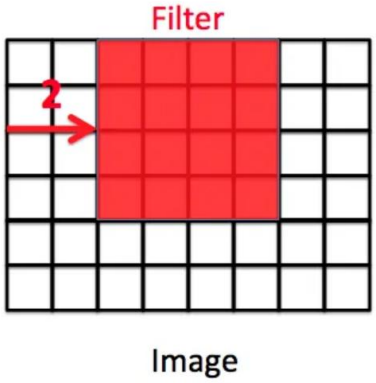
**Stride = 1:** the filter moves one pixel at a time.

- Captures more spatial information.
- The output feature map is comparatively large.
- Computation can be higher.

**Stride = 2:** the filter moves two pixels at a time.

- The output feature map is smaller (roughly half the height and width).
- Computation is lower.
- Some spatial detail may be lost.

If stride = 2, the filter will move two pixels.




*With stride = 2 the filter moves two pixels at a time.*

Stride 2 is sometimes used instead of pooling to downsample a feature map. The output height and width are calculated separately, with the same formula (K is the kernel size, P the padding, S the stride, and the brackets mean rounding down):

```
H_out = floor( (H + 2P - K) / S ) + 1
W_out = floor( (W + 2P - K) / S ) + 1

Example: input 6 x 8, K = 3, P = 1, S = 2
H_out = floor((6 + 2 - 3) / 2) + 1 = floor(2.5) + 1 = 3
W_out = floor((8 + 2 - 3) / 2) + 1 = floor(3.5) + 1 = 4      ->  output is 3 x 4

import numpy as np

def conv2d(x, k, stride=1, pad=0):
    x = np.pad(x, pad)                                  # zero padding on all sides
    n = (x.shape[0] - k.shape[0]) // stride + 1          # output size formula
    out = np.zeros((n, n))
    for i in range(n):
        for j in range(n):
            patch = x[i*stride:i*stride+k.shape[0], j*stride:j*stride+k.shape[1]]
            out[i, j] = np.sum(patch * k)
    return out

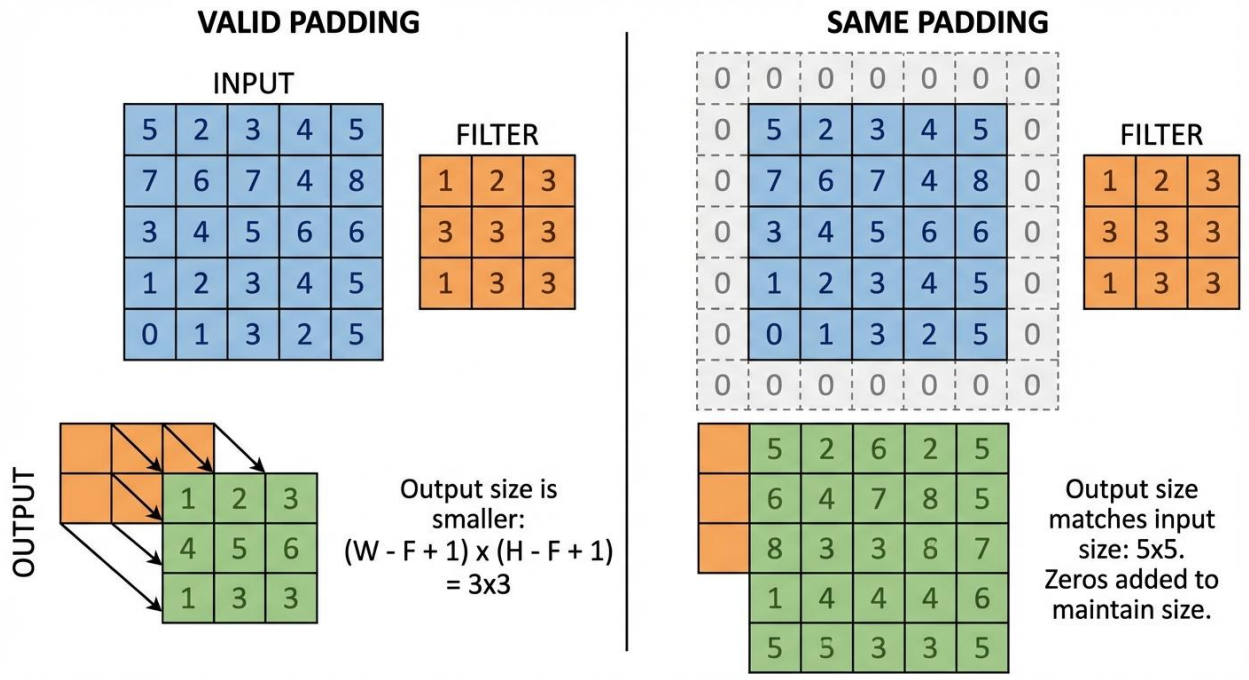
x = np.array([[5, 2, 3, 4, 5],
              [7, 6, 7, 4, 8],
              [3, 4, 5, 6, 6],
              [1, 2, 3, 4, 5],
              [0, 1, 3, 2, 5]])
k = np.array([[1, 2, 3],
              [3, 3, 3],
              [1, 3, 3]])

print(conv2d(x, k, stride=1, pad=0))      # valid  -> 3x3
print(conv2d(x, k, stride=1, pad=1))      # same   -> 5x5
print(conv2d(x, k, stride=2, pad=0).shape)  # (2, 2)
print(conv2d(x, k, stride=2, pad=1).shape)  # (3, 3)
```

Output of the first two calls:

```
[[108. 108. 124.]
 [ 92. 100. 120.]
 [ 56.  75.  95.]]

[[ 60.  76.  66.  79.  55.]
 [ 76. 108. 108. 124.  74.]
 [ 62.  92. 100. 120.  75.]
 [ 30.  56.  75.  95.  62.]
 [ 11.  26.  38.  56.  35.]]
```




*Valid vs. same padding with a 5x5 input and a 3x3 filter (illustration).*

Note: in this last illustration the numbers inside the green output grids are placeholders and are not the real convolution results for the input and filter shown. The sizes (3x3 for valid, 5x5 for same) are correct. Real values for that exact input and filter are in the printed output above.

**In Keras and PyTorch:**

```
# Keras
layers.Conv2D(32, (3, 3), padding="valid", strides=1)   # default: output shrinks
layers.Conv2D(32, (3, 3), padding="same",  strides=1)   # output keeps input size
layers.Conv2D(32, (3, 3), padding="same",  strides=2)   # output is about half the size

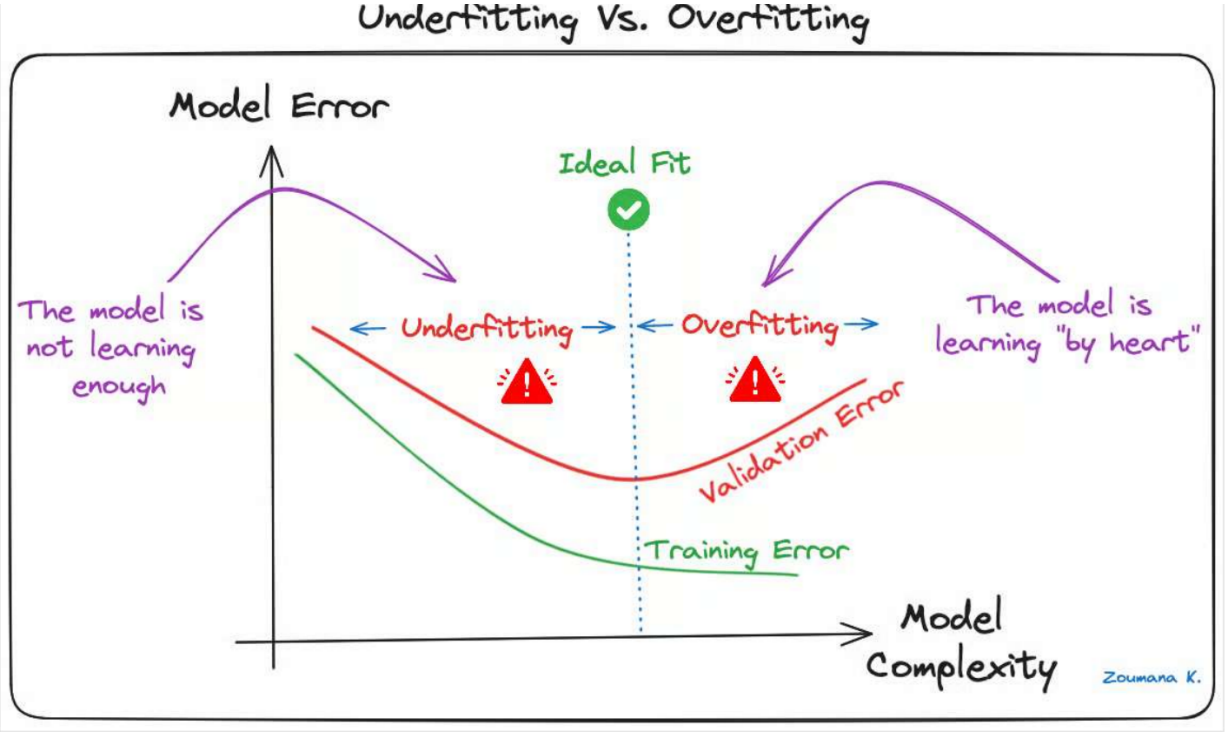
# PyTorch
nn.Conv2d(1, 32, kernel_size=3, padding=0, stride=1)    # valid
nn.Conv2d(1, 32, kernel_size=3, padding=1, stride=1)    # same (for a 3x3 kernel)
nn.Conv2d(1, 32, kernel_size=3, padding=1, stride=2)    # downsample by about 2
nn.Conv2d(1, 32, kernel_size=3, padding=1, padding_mode="reflect")
```

### Fully connected layers

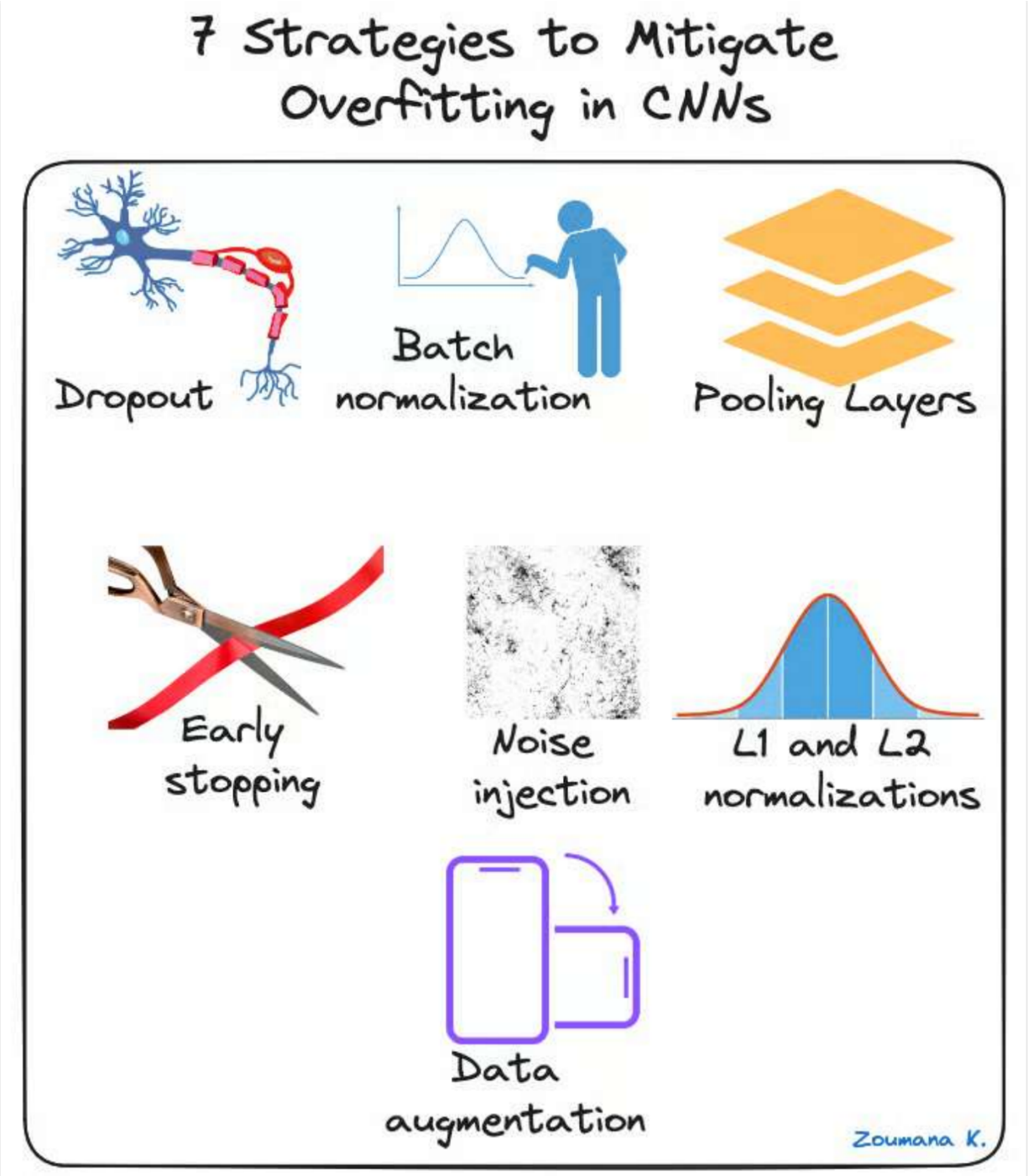
The last pooling layer flattens its feature map into a 1-D vector for the fully connected layers, which use ReLU for non-linearity. Finally, a softmax layer produces a probability for each output label, and the label with the highest probability is the prediction.

## 4. Overfitting and Regularization in CNNs

Overfitting happens when a model learns the training data too well, including its noise, and then performs badly on new data. It shows up as much higher accuracy on training data than on validation or test data. CNNs are especially susceptible because of their capacity for high complexity.




*Underfitting vs. overfitting.*

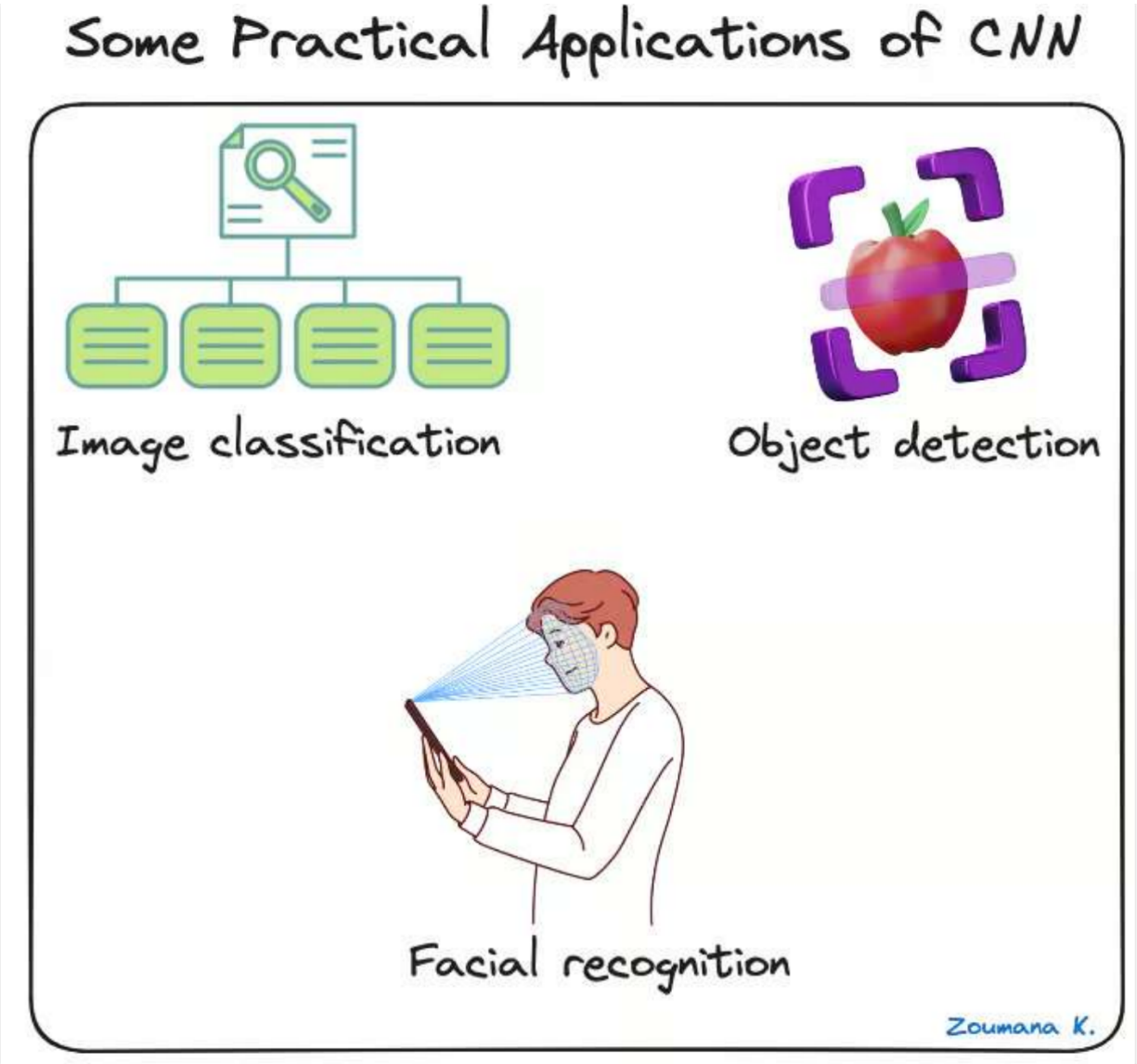



*7 strategies to mitigate overfitting in CNNs.*

- **Dropout:** randomly drops neurons during training, forcing the others to learn new features.
- **Batch normalization:** normalizes activations, reducing overfitting somewhat and stabilizing/speeding up training.
- **Pooling layers:** reduce spatial dimensions to give a more abstract representation.

- **Early stopping:** stop training when validation error stops improving.
- **Noise injection:** add noise to inputs or hidden-layer outputs during training.
- **L1 and L2 regularization:** penalize large weights; L1 encourages sparse weights, L2 (weight decay) encourages small weights.
- **Data augmentation:** enlarge the dataset with random rotations, scaling, flipping, or cropping.

## 5. Practical Applications of CNNs




*Some practical applications of CNNs.*

- **Image classification:** assigning images to predefined categories, e.g. automatic photo organization.

- **Object detection:** identifying and locating multiple objects, e.g. shelf scanning in retail.
- **Facial recognition:** e.g. access control in security systems.

# 6. Popular CNN Architectures

- **LeNet-5 (1998):** one of the first CNNs, built for handwritten digits.
- **AlexNet (2012):** won ImageNet and popularized deep CNNs with GPU training.
- **VGGNet (2014):** showed that deeper networks with small 3x3 filters improve accuracy.
- **GoogLeNet/Inception (2014):** inception modules with parallel filter sizes.
- **ResNet (2015):** skip connections enabling networks with 100+ layers.
- **EfficientNet (2019):** compound scaling of depth, width, and resolution.
- **ConvNeXt (2022):** a modernized CNN that competes with Vision Transformers.

Vision Transformers (ViTs) have emerged as strong alternatives since 2020, but CNNs remain widely used for their efficiency, lower data requirements, and maturity in production.

# 7. Deep Learning Frameworks

**TensorFlow** (Google, 2015) is an open-source framework with tools for development and deployment. **Keras** is its official high-level API (since 2.0) for rapid experimentation. **PyTorch** (Meta, 2017) is known for its dynamic computational graph, Pythonic syntax, and strong research community.

| | TensorFlow | PyTorch | Keras |
|---|---|---|---|
| **API level** | High and low | Low | High |
| **Syntax** | Harder to use | Pythonic, intuitive | Simple, concise |
| **Datasets** | Large, high performance | Large, high performance | Smaller datasets |
| **Debugging** | Difficult | Good capabilities | Rarely needed |
| **Pretrained models** | Yes | Yes | Yes |
| **Popularity** | Second of the three | Most used for research, growing in production | TensorFlow's official high-level API |
| **Speed** | Fast | Fast | Same as TensorFlow |
| **Written in** | C++, CUDA, Python | C++, Python | Python |

# 8. Backpropagation in CNNs

Backpropagation is how a CNN learns: after a forward pass produces a prediction, the loss is differentiated with respect to every trainable parameter using the chain rule, and each parameter is nudged against its gradient. This section walks through a tiny CNN so every step can be followed by hand.

## 8.1 The sample network

A 6x6 image goes through one 3x3 filter (convolution, stride 1), a ReLU, 2x2 max pooling (stride 2), flattening, and one fully connected output neuron with a sigmoid. The task is binary classification, so the loss is binary cross-entropy. Shapes: 6x6 -> 4x4 -> 4x4 -> 2x2 -> 4 values -> 1 output.

**Trainable parameters: 15 in total.** ReLU, pooling and flattening have no parameters.

| Layer | Parameters | Count |
|---|---|---|
| Convolution (3x3 filter) | 9 filter weights W, 1 bias b | 10 |
| Fully connected (4 -> 1) | 4 weights w, 1 bias b_out | 5 |
| **Total** | | **15** |

The network splits logically into a **CNN part** (convolution, ReLU, pooling: feature extraction) and an **ANN part** (flatten, fully connected, sigmoid: classification). Backpropagation is split the same way, starting from the ANN part at the output and working back toward the input.

## 8.2 Forward propagation

```
Z[i,j] = sum over m,n of  W[m,n] * X[i+m, j+n]  +  b        (convolution, 4x4)
A      = ReLU(Z) = max(0, Z)                                  (4x4)
P      = maxpool_2x2(A)                                       (2x2)
f      = flatten(P) = [f1, f2, f3, f4]
z_out  = w1*f1 + w2*f2 + w3*f3 + w4*f4 + b_out
y_hat  = sigmoid(z_out) = 1 / (1 + exp(-z_out))
L      = -[ y*log(y_hat) + (1 - y)*log(1 - y_hat) ]           (binary cross-entropy)
```

## 8.3 Backpropagation through the ANN part

Apply the chain rule from the loss back to the output neuron. The derivative of the loss with respect to the prediction, multiplied by the sigmoid derivative, simplifies neatly:

```
dL/dy_hat     = -y / y_hat + (1 - y) / (1 - y_hat)
dy_hat/dz_out = y_hat * (1 - y_hat)

dL/dz_out = dL/dy_hat * dy_hat/dz_out = y_hat - y          <- the key simplification

dL/dw_i   = dL/dz_out * dz_out/dw_i = (y_hat - y) * f_i     (i = 1..4)
dL/db_out = (y_hat - y)
```

So each output weight is updated in proportion to the error (y_hat - y) times the feature it multiplies, and the bias in proportion to the error alone.

## 8.4 Backpropagation through the CNN part

The error now has to travel backward through flatten, max pooling, ReLU, and the convolution. Each step has a simple rule:

- **Fully connected input:** dL/df_i = (y_hat - y) * w_i.
- **Flatten (backward):** reshape the 4 values back to the 2x2 shape of the pooled output, giving dL/dP.
- **Max pooling (backward):** in each 2x2 window the gradient goes only to the position that held the maximum; the other three positions get 0. This produces a 4x4 gradient dL/dA, mostly zeros.
- **ReLU (backward):** dL/dZ = dL/dA where Z > 0, and 0 where Z <= 0 (the ReLU derivative is 1 or 0).
- **Convolution (backward):** the filter gradient is the input correlated with the incoming gradient. Each filter weight collects the gradient from every position where it was used (weight sharing).

```
dL/dW[m,n] = sum over i,j of  dL/dZ[i,j] * X[i+m, j+n]       (3x3 result)
dL/db      = sum over i,j of  dL/dZ[i,j]
```

## 8.5 Parameter update (gradient descent)

```
W     <- W     - lr * dL/dW          w     <- w     - lr * dL/dw
b     <- b     - lr * dL/db          b_out <- b_out - lr * dL/db_out
```

Repeating forward pass, backward pass, and update over many examples is training. Frameworks such as TensorFlow and PyTorch compute all of these gradients automatically (autograd), but understanding the mechanics explains why pooling and ReLU pass gradients only through selected positions, and why convolution weights are shared. Section 9.2 implements this exact network in NumPy and checks the gradients numerically.

# 9. Hands-On: Building CNNs in Code

This section turns the theory above into code. It starts with the convolution and pooling operations from scratch in NumPy, then builds a full digit classifier in Keras and in PyTorch using the MNIST dataset (28x28 grayscale digits, 10 classes).

## Setup

```
pip install numpy matplotlib tensorflow torch torchvision
```

## 9.1 Convolution and max pooling from scratch (NumPy)

This reproduces the worked example from Section 3: the 6x6 pixel matrix, the 3x3 kernel, stride 1, then 2x2 max pooling with stride 2.

```
import numpy as np

def conv2d(image, kernel, stride=1):
    kh, kw = kernel.shape
    out_h = (image.shape[0] - kh) // stride + 1
    out_w = (image.shape[1] - kw) // stride + 1
    out = np.zeros((out_h, out_w))
    for i in range(out_h):
        for j in range(out_w):
            patch = image[i*stride:i*stride+kh, j*stride:j*stride+kw]
            out[i, j] = np.sum(patch * kernel)      # element-wise multiply, then sum
    return out

def max_pool(fmap, size=2, stride=2):
    out_h = (fmap.shape[0] - size) // stride + 1
    out_w = (fmap.shape[1] - size) // stride + 1
    out = np.zeros((out_h, out_w))
    for i in range(out_h):
        for j in range(out_w):
            out[i, j] = fmap[i*stride:i*stride+size, j*stride:j*stride+size].max()
    return out

def relu(x):
    return np.maximum(0, x)

image = np.array([[0, 0, 0,  0, .2, 0 ],
                  [0, 0, 0,  0, .2, .3],
                  [0, 0, .1, 0, .5, .2],
                  [0, .2, 0, .2, .2, .6],
                  [0, .2, 0, 0, .9, .7],
                  [0, 0, 0,  0, 0,  0 ]])
kernel = np.array([[1, 0, 1],
                   [0, 1, 0],
                   [1, 0, 1]])

fmap = relu(conv2d(image, kernel))
print(fmap.round(2))
print(max_pool(fmap).round(2))
```

Output (verified by running the code):

```
[[0.1 0.  0.8 0.4]
 [0.  0.5 0.4 1.6]
```

```
 [0.3 0.2 1.7 1.1]
 [0.2 0.4 0.2 1.7]]

[[0.5 1.6]
 [0.4 1.7]]
```

Note: the original article's figure shows 1.0, 1.1 and a pooled result of [[0.5, 1.6], [0, 0.5]] in a few cells. Recomputing the same inputs gives the values above, so the printed figure appears to contain small arithmetic slips. Everything else (first value 0.1, bottom-row 0.4, the 4x4 output size) matches.

Output size formula: **out = floor((n - k) / stride) + 1**, where n is the input size and k the kernel size. A 6x6 input with a 3x3 kernel and stride 1 gives 4x4; pooling 4x4 with a 2x2 window and stride 2 gives 2x2.

## 9.2 Backpropagation from scratch (NumPy)

This implements the network from Section 8 and its backward pass by hand. A numerical gradient check (finite differences) confirms the derivatives are correct, then a few gradient-descent steps show the loss falling.

```
import numpy as np
np.random.seed(0)

def sigmoid(z):
    return 1 / (1 + np.exp(-z))

def forward(X, W, b, w, b_out):
    Z = np.zeros((4, 4))
    for i in range(4):
        for j in range(4):
            Z[i, j] = np.sum(X[i:i+3, j:j+3] * W) + b      # convolution (stride 1)
    A = np.maximum(0, Z)                                    # ReLU
    P = A.reshape(2, 2, 2, 2).max(axis=(1, 3))              # 2x2 max pooling, stride 2
    f = P.flatten()                                         # flatten -> 4 values
    y_hat = sigmoid(np.dot(w, f) + b_out)                   # fully connected + sigmoid
    return Z, A, P, f, y_hat

def bce(y, y_hat):
    return -(y * np.log(y_hat) + (1 - y) * np.log(1 - y_hat))

def backward(X, y, cache, w):
    Z, A, P, f, y_hat = cache
    # ---- ANN part ----
    dz_out = y_hat - y                # dL/dz_out  (sigmoid + BCE)
    dw = dz_out * f                   # dL/dw
    db_out = dz_out                   # dL/db_out
    df = dz_out * w                   # dL/df
    # ---- CNN part ----
    dP = df.reshape(2, 2)             # un-flatten
    dA = np.zeros_like(A)             # max-pool backward: gradient to the max position only
    for i in range(2):
        for j in range(2):
            win = A[2*i:2*i+2, 2*j:2*j+2]
            m, n = np.unravel_index(np.argmax(win), win.shape)
            dA[2*i+m, 2*j+n] = dP[i, j]
    dZ = dA * (Z > 0)                 # ReLU backward
    dW = np.zeros((3, 3))             # convolution backward
    for m in range(3):
```

```
        for n in range(3):
            dW[m, n] = np.sum(dZ * X[m:m+4, n:n+4])
    db = dZ.sum()
    return dW, db, dw, db_out

# Toy data and parameters (15 trainable values in total)
X = np.random.rand(6, 6)
y = 1
W = np.random.randn(3, 3) * 0.5
b = 0.1
w = np.random.randn(4) * 0.5
b_out = 0.0

# Training: forward -> backward -> update
lr = 0.5
for step in range(6):
    cache = forward(X, W, b, w, b_out)
    print(f"step {step}: loss={bce(y, cache[4]):.4f}  y_hat={cache[4]:.4f}")
    dW, db, dw, db_out = backward(X, y, cache, w)
    W -= lr * dW;  b -= lr * db
    w -= lr * dw;  b_out -= lr * db_out
```

Gradient check: add this **before** the training loop. It compares the analytic gradients with finite differences.

```
def num_grad(loss_fn, arr, eps=1e-6):
    g = np.zeros_like(arr, dtype=float)
    for idx in np.ndindex(*arr.shape):
        old = arr[idx]
        arr[idx] = old + eps;  lp = loss_fn()
        arr[idx] = old - eps;  lm = loss_fn()
        arr[idx] = old
        g[idx] = (lp - lm) / (2 * eps)
    return g

cache = forward(X, W, b, w, b_out)
dW, db, dw, db_out = backward(X, y, cache, w)

L = lambda: bce(y, forward(X, W, b, w, b_out)[4])
print(np.abs(dW - num_grad(L, W)).max())     # filter weights
print(np.abs(dw - num_grad(L, w)).max())     # output weights
```

Gradient check result: comparing the analytic gradient with (L(p+eps) - L(p-eps)) / (2*eps) for every parameter gave a maximum difference of about 1e-10 for the filter, the filter bias, the output weights and the output bias, so the backward pass is correct. Training output (verified by running the code):

```
step 0: loss=0.5714  y_hat=0.5647
step 1: loss=0.0990  y_hat=0.9058
step 2: loss=0.0500  y_hat=0.9512
step 3: loss=0.0311  y_hat=0.9694
step 4: loss=0.0219  y_hat=0.9783
step 5: loss=0.0166  y_hat=0.9835
```

The learning rate here is large (0.5) because this is a single toy example; for real training use values like 1e-3 with Adam.

## 9.3 Digit classifier in Keras (TensorFlow)

This model uses all four components (convolution, ReLU, pooling, fully connected + softmax) and applies the regularization techniques from Section 4: data augmentation, batch normalization, L2 weight decay, dropout, and early stopping.

```
import numpy as np
import tensorflow as tf
from tensorflow.keras import layers, models, regularizers, callbacks

# 1. Load and preprocess data
(x_train, y_train), (x_test, y_test) = tf.keras.datasets.mnist.load_data()
x_train = x_train[..., np.newaxis].astype("float32") / 255.0   # (60000, 28, 28, 1)
x_test  = x_test[..., np.newaxis].astype("float32") / 255.0

# 2. Build the model
model = models.Sequential([
    layers.Input(shape=(28, 28, 1)),

    # Data augmentation (only active during training)
    layers.RandomRotation(0.05),
    layers.RandomZoom(0.1),

    # Block 1: convolution -> batch norm -> ReLU -> pooling
    layers.Conv2D(32, (3, 3)),            # 28x28 -> 26x26x32
    layers.BatchNormalization(),
    layers.Activation("relu"),
    layers.MaxPooling2D((2, 2)),          # -> 13x13x32

    # Block 2
    layers.Conv2D(64, (3, 3)),            # -> 11x11x64
    layers.BatchNormalization(),
    layers.Activation("relu"),
    layers.MaxPooling2D((2, 2)),          # -> 5x5x64

    # Classifier
    layers.Flatten(),                     # -> 1600
    layers.Dense(128, activation="relu",
                 kernel_regularizer=regularizers.l2(1e-4)),   # L2 regularization
    layers.Dropout(0.5),                  # dropout
    layers.Dense(10, activation="softmax"),
])
model.summary()

# 3. Compile
model.compile(optimizer="adam",
              loss="sparse_categorical_crossentropy",
              metrics=["accuracy"])

# 4. Train with early stopping
early_stop = callbacks.EarlyStopping(monitor="val_loss", patience=3,
                                     restore_best_weights=True)
history = model.fit(x_train, y_train,
                    validation_split=0.1,
                    epochs=30, batch_size=128,
                    callbacks=[early_stop])

# 5. Evaluate and predict
test_loss, test_acc = model.evaluate(x_test, y_test, verbose=0)
print(f"Test accuracy: {test_acc:.4f}")
```

```
probs = model.predict(x_test[:5])
print("Predicted:", probs.argmax(axis=1), "Actual:", y_test[:5])

model.save("mnist_cnn.keras")
```

## Plot training vs. validation curves to spot overfitting

```
import matplotlib.pyplot as plt

plt.plot(history.history["accuracy"], label="train")
plt.plot(history.history["val_accuracy"], label="validation")
plt.xlabel("Epoch"); plt.ylabel("Accuracy"); plt.legend(); plt.show()
# A growing gap between the two curves is the classic sign of overfitting.
```

## 9.4 The same model in PyTorch

PyTorch is more explicit: you define the model class and write the training loop yourself.

```
import torch
import torch.nn as nn
import torch.nn.functional as F
from torch.utils.data import DataLoader, random_split
from torchvision import datasets, transforms

device = "cuda" if torch.cuda.is_available() else "cpu"

# 1. Data (with augmentation for the training set)
train_tf = transforms.Compose([
    transforms.RandomAffine(degrees=5, scale=(0.9, 1.1)),
    transforms.ToTensor(),
])
test_tf = transforms.ToTensor()

full_train = datasets.MNIST("data", train=True, download=True, transform=train_tf)
test_set   = datasets.MNIST("data", train=False, download=True, transform=test_tf)
train_set, val_set = random_split(full_train, [54000, 6000])

train_loader = DataLoader(train_set, batch_size=128, shuffle=True)
val_loader   = DataLoader(val_set, batch_size=256)
test_loader  = DataLoader(test_set, batch_size=256)

# 2. Model
class CNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.conv1 = nn.Conv2d(1, 32, kernel_size=3)    # 28 -> 26
        self.bn1   = nn.BatchNorm2d(32)
        self.conv2 = nn.Conv2d(32, 64, kernel_size=3)   # 13 -> 11
        self.bn2   = nn.BatchNorm2d(64)
        self.pool  = nn.MaxPool2d(2)                    # halves height and width
        self.fc1   = nn.Linear(64 * 5 * 5, 128)         # 5x5x64 = 1600 features
        self.drop  = nn.Dropout(0.5)
        self.fc2   = nn.Linear(128, 10)

    def forward(self, x):
        x = self.pool(F.relu(self.bn1(self.conv1(x))))  # -> 13x13x32
        x = self.pool(F.relu(self.bn2(self.conv2(x))))  # -> 5x5x64
        x = torch.flatten(x, 1)
        x = self.drop(F.relu(self.fc1(x)))
        return self.fc2(x)       # raw logits; softmax is built into the loss
```

```
model = CNN().to(device)
criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=1e-3, weight_decay=1e-4)  # L2

def evaluate(loader):
    model.eval()
    correct = total = 0
    loss_sum = 0.0
    with torch.no_grad():
        for x, y in loader:
            x, y = x.to(device), y.to(device)
            out = model(x)
            loss_sum += criterion(out, y).item() * len(y)
            correct += (out.argmax(1) == y).sum().item()
            total += len(y)
    return loss_sum / total, correct / total

# 3. Training loop with early stopping
best_val, patience, bad = float("inf"), 3, 0
for epoch in range(30):
    model.train()
    for x, y in train_loader:
        x, y = x.to(device), y.to(device)
        optimizer.zero_grad()
        loss = criterion(model(x), y)
        loss.backward()
        optimizer.step()

    val_loss, val_acc = evaluate(val_loader)
    print(f"Epoch {epoch+1}: val_loss={val_loss:.4f}  val_acc={val_acc:.4f}")

    if val_loss < best_val:
        best_val, bad = val_loss, 0
        torch.save(model.state_dict(), "best_cnn.pt")
    else:
        bad += 1
        if bad >= patience:
            print("Early stopping.")
            break

# 4. Test with the best weights
model.load_state_dict(torch.load("best_cnn.pt"))
_, test_acc = evaluate(test_loader)
print(f"Test accuracy: {test_acc:.4f}")
```

## 9.5 Transfer learning (fine-tuning a pre-trained CNN)

As noted in Section 1, pre-trained architectures can be adapted to new tasks with little data. Here, a ResNet50 trained on ImageNet is reused as a frozen feature extractor, and only a new classification head is trained (Keras).

```
from tensorflow.keras import layers, models
from tensorflow.keras.applications import ResNet50

NUM_CLASSES = 5   # set to the number of classes in your own dataset

base = ResNet50(weights="imagenet", include_top=False, input_shape=(224, 224, 3))
base.trainable = False                       # freeze the pre-trained convolutional layers

model = models.Sequential([
```

```
    base,
    layers.GlobalAveragePooling2D(),
    layers.Dropout(0.3),
    layers.Dense(NUM_CLASSES, activation="softmax"),
])
model.compile(optimizer="adam", loss="sparse_categorical_crossentropy", metrics=["accuracy"])

# model.fit(train_ds, validation_data=val_ds, epochs=10)
# Optional second stage: unfreeze the top layers of `base` and retrain with a very low
# learning rate (e.g. 1e-5) for fine-tuning.
```

### Shape cheat-sheet and tips

- Each 3x3 convolution without padding trims 2 pixels from height and width; each 2x2 pooling halves them (28 -> 26 -> 13 -> 11 -> 5).
- Flatten size = height x width x channels of the last feature map (5 x 5 x 64 = 1600), which is what the first Dense/Linear layer must accept.
- Use `padding="same"` (Keras) or `padding=1` (PyTorch) to keep the spatial size unchanged after a 3x3 convolution.
- Keras with `softmax` uses `sparse_categorical_crossentropy`; PyTorch's `CrossEntropyLoss` expects raw logits, so do not add softmax in the model.
- If validation accuracy stalls well below training accuracy, add more augmentation or dropout, or reduce model size.
- Train on a GPU for anything larger than MNIST (Google Colab offers one for free).

Note: the Keras and PyTorch snippets were not executed in the environment used to prepare this document (TensorFlow/PyTorch were not installed there and MNIST needs a download). The layer shapes were checked by hand, and the NumPy example was run. Typical expectation for this model on MNIST is roughly 99% test accuracy.

## Summary

A CNN stacks convolution + ReLU + pooling blocks to extract increasingly abstract features, then uses fully connected layers with softmax to classify. Overfitting is controlled with dropout, batch normalization, weight penalties, data augmentation, and early stopping. Backpropagation applies the chain rule layer by layer: the output error (y_hat - y) flows back through the fully connected layer, is routed by max pooling and ReLU, and becomes the filter gradient in the convolution. TensorFlow/Keras and PyTorch compute all of this automatically, as shown in the code above.